



Grupo: Integra Consulting

Análisis de Requisitos - Fixture Mundial 2030

Curso: Ingeniería de Datos II		Aula: B101	Turno: Noche
Integrantes:			
1.	Axel Fischbein		LU: 1221914
2.	Mena Mohamed David		LU: 1128374
3.	Ocampo Denise		LU: 1130334
Profesor: Joaquín Salas			
Fecha: 14/8/2026		Cuatrimestre: 2	

1. Datos que debe manejar el fixture

El Fixture del Mundial 2030 es una plataforma interactiva de pronósticos deportivos. Además de informar sobre las selecciones, los jugadores y los partidos, permite que cada usuario cargue sus pronósticos, acumule puntos y consulte su puntaje total y su posición en el ranking. Para el análisis consideramos los datos definitivos que necesita la plataforma, evaluados según su propósito, volumen, crecimiento, patrón de acceso y criticidad operativa.

Distinguimos volumen (cuántos datos hay) de crecimiento (a qué velocidad aparecen datos nuevos), porque son dos presiones distintas sobre el motor. El volumen encarece las lecturas y el mantenimiento de índices, mientras que el crecimiento encarece las escrituras.

Caracterización de los datos:

Dato	Qué representa	Volumen	Crecimiento	Patrón de acceso	Operación típica	Criticidad
Equipos	Selección, país, técnico, escudo, ranking FIFA y plantel.	64	Nulo	Lectura » escritura	Ver la ficha de una selección.	Media
Jugadores	Perfil, dorsal, posición y estadísticas acumuladas.	1.500+	Nulo	Lectura » escritura	Ver la ficha de un jugador.	Media
Partidos	Fecha, hora, sede, país anfitrión, fase, equipos, estado y resultado.	127 en 6 países	Nulo	Lectura masiva; escritura acotada	Consultar el fixture o el resultado de un partido.	Alta
Usuarios	Perfil, favoritos y preferencias.	Millones (2-3 M simultáneos)	Constante	Lectura y escritura	Abrir o modificar el perfil.	Alta
Sesiones	Token, usuario, dispositivo y vencimiento.	2-3 M activas	Alta rotación; crece durante los partidos	Lectura muy frecuente	Validar un token en cada request.	Alta
Pronósticos	Marcador previsto por usuario y partido, y puntos obtenidos.	Cientos de millones	A ráfagas: pico antes de cada cierre	Escritura y lectura	Guardar el pronóstico de un partido.	Alta
Puntaje / ranking	Puntos totales y posición de cada participante.	Millones de contadores	Salto al cerrar cada partido	Lectura muy frecuente; escritura por lotes	Ver el Top 100 o la posición propia.	Alta
Eventos	Goles, tarjetas, cambios, faltas y asistencias.	1.000+ por partido	En vivo, en ráfagas	Escritura continua y lectura	Registrar un gol; ver el minuto a minuto.	Alta en vivo
Comentarios	Mensajes y reacciones de los usuarios durante los partidos.	1 M+ por partido	Muy alto, con picos	Escritura predominante	Publicar y leer los últimos comentarios.	Media
Estadísticas	Posesión, tiros, pases y otras métricas con marca temporal.	10 M+ puntos por partido	Tiempo real, continuo	Escritura masiva y agregación	Consultar la posesión entre los minutos 30 y 45.	Media
Auditoría	Historial de operaciones de la plataforma.	Petabytes	Continuo; nunca decrece	Escritura continua; lectura ocasional	Buscar las operaciones de un usuario en una fecha.	Media

Consideramos de criticidad alta a los datos cuya falla afectaría el acceso, la participación o los resultados del juego. Los volúmenes generales provienen del apéndice del trabajo práctico; los de pronósticos y ranking son estimaciones propias según la participación esperada. Nos apartamos del apéndice en un

punto. Allí la auditoría figura con acceso de lectura, pero su carga real es de escritura continua y lectura esporádica.

2. Análisis de una solución relacional

Una base relacional puede almacenar todos estos datos, y para equipos, jugadores y partidos sería incluso la opción más razonable: volumen fijo, relaciones claras e integridad garantizada por claves foráneas. El problema no es que SQL sea inadecuado en general, sino pretender que un único motor responda igual a cargas de trabajo opuestas. Habría que sostener al mismo tiempo más de 100.000 solicitudes por segundo, una latencia menor a 100 ms, un 99,99 % de uptime y la distribución en varias regiones.

El límite es de fondo. SQL ofrece garantías ACID y escala sobre todo agrandando el servidor. También se puede escalar sumando máquinas, con réplicas de lectura y particionado, pero las réplicas introducen un retraso de sincronización y el particionado rompe los JOIN y las transacciones que cruzan particiones, con lo cual se termina perdiendo la ventaja por la que se había elegido SQL.

El esquema relacional de referencia se incluye en el **Anexo A**, con el detalle completo en el archivo fixture2030.dbml. Respecto de la primera versión corregimos tres omisiones. PRONOSTICO lleva UNIQUE (id_usuario, id_partido) para garantizar una sola predicción por usuario y partido, SESSION.token es único, y USUARIO, EQUIPO y JUGADOR incluyen los atributos declarados en la sección 1 (preferencias y equipo favorito, técnico y escudo, dorsal).

Análisis por grupo de datos:

Dato	Cómo se almacenaría en SQL	Problema específico	Impacto en rendimiento y escalabilidad
Equipos y jugadores	Tablas EQUIPO y JUGADOR relacionadas; la posición como atributo del jugador.	La ficha completa exige 3 JOIN (JUGADOR ↔ EQUIPO ↔ EVENTO ↔ PARTIDO) más la agregación de goles y tarjetas.	Bajo. Con 1.500 filas todo entra en memoria, así que SQL sigue siendo competitivo para este dato.
Partidos, resultados y eventos	PARTIDO relacionado con los equipos local y visitante, y con RESULTADO y EVENTO.	El minuto a minuto encadena 4 tablas y, en vivo, se insertan eventos mientras miles de usuarios leen la misma fila.	El volumen es trivial, pero la ráfaga de INSERT compite con las lecturas sobre el mismo índice.
Usuarios	USUARIO relacionada con pronósticos, puntaje, sesiones, comentarios y auditoría.	Con 2-3 M conectados hacen falta índices, réplicas, particionado y caché para sostener la latencia.	El particionado por id_usuario rompe los JOIN con PARTIDO, que se particiona por otro criterio.
Sesiones	SESSION con el usuario, el token y su vencimiento.	Cada request valida un token, lo que da unas 100.000 lecturas por segundo sobre una tabla de rotación permanente.	Los DELETE de sesiones vencidas fragmentan el índice y compiten con las lecturas.
Pronósticos	Una fila por usuario y partido, con el marcador y los puntos obtenidos.	En los minutos previos al cierre millones de usuarios escriben a la vez, con decenas de miles de INSERT por segundo sobre un índice único.	Se genera contención en el índice y en el registro de transacciones, y cientos de millones de filas encarecen cada reconstrucción.
Puntaje y ranking	PUNTAJE con los puntos totales; la posición se calcula ordenando.	El Top 100 es barato con un índice, pero la posición propia exige contar cuántos usuarios superan a uno, y se consulta en cada visita.	Al cerrar un partido se actualizan millones de filas y el índice de puntos se reescribe casi por completo.
Comentarios	COMENTARIO relacionado con el usuario y el partido.	Más de un millón por partido con INSERT simultáneos, y el índice debe mantenerse en pleno pico.	El nodo maestro es el cuello de botella y no hay forma de repartir la escritura sin particionar.
Estadísticas	ESTADISTICA con partido, instante, tipo de métrica y valor.	Consultar un intervalo obliga a agregar sobre decenas de millones de filas por partido.	La agregación temporal no aprovecha la fila como unidad y no existe una reducción de resolución nativa.

3. Propuesta NoSQL y matriz de decisión

Criterio de la propuesta

Cuando los datos dejan de estar en un solo servidor aparece una tensión que en una base única no existe. Si una máquina queda incomunicada, o el sistema espera a que todas las copias estén sincronizadas y mientras tanto no responde, o responde igual y acepta que durante unos segundos algunos usuarios lean información un poco atrasada. Las bases NoSQL suelen inclinarse por lo segundo y a cambio ganan algo que a SQL le cuesta, que es escalar sumando máquinas. Los datos se reparten entre nodos iguales según una clave de partición y agregar capacidad es agregar nodos. Lo que se resigna también es concreto, y son las transacciones que cruzan entidades, los JOIN, el esquema garantizado y las consultas que no se previeron al diseñar.

Por eso la elección no puede ser la misma para todo. Los pronósticos y el puntaje definen los resultados del juego, así que ahí preferimos esperar a que la escritura quede confirmada en la mayoría de las copias aunque cueste unos milisegundos más. En cambio los comentarios, las estadísticas y los eventos toleran unos segundos de retraso con tal de seguir disponibles durante los picos. Con ese criterio distribuimos los datos entre MongoDB, Neo4j, Redis, Cassandra, IRIS e InfluxDB, que son las tecnologías previstas para el trabajo práctico.

Matriz de decisión:

Dato	Modelo elegido	Razón y ventaja frente a SQL	Alternativa descartada	Trade-off (qué se resigna)
Equipos	Documental (MongoDB)	La ficha con el plantel se recupera como un único documento, sin reconstruirla con JOIN.	SQL, válido por el volumen fijo, pero exige combinar tablas para armar la ficha completa.	Se duplican datos del plantel y hay que actualizarlos en más de un lugar.
Jugadores	Documental (MongoDB)	Admite perfiles con atributos variables según la posición y permite consultar la ficha como una unidad.	SQL, que da integridad, pero con un esquema rígido que complica los atributos que varían.	Al no haber claves foráneas, la consistencia con el equipo queda a cargo de la aplicación.
Partidos	Documental (MongoDB) + proyección en grafo (Neo4j)	El fixture y el resultado son un documento, y la proyección en grafo habilita recorridos como los rivales que enfrentó un equipo o el camino jugador-partido-equipo, que en SQL serían JOIN encadenados.	Usar solo grafo, algo que con 127 partidos y relaciones simples no se justifica como almacén principal.	Quedan dos representaciones del mismo dato y hay que sincronizarlas después de cada cambio.
Eventos	Columnas anchas (Cassandra) para la ingesta + Neo4j para el análisis	Absorbe la ráfaga en vivo repartiendo la escritura por partido, y el grafo conecta gol, jugador, asistencia y equipo con relaciones explícitas.	Usar solo Neo4j, que es un motor pensado para consultar relaciones y no para ingesta masiva.	Hay un retraso entre lo que se escribe y lo que ya se puede recorrer en el grafo.
Usuarios	Documental (MongoDB) como fuente de verdad + Redis como caché	El perfil vive en un documento y Redis sirve los datos calientes con una latencia mucho menor.	Usar solo Redis, que es una caché y no el almacén persistente del perfil.	Hay que invalidar la caché, porque el perfil puede leerse desactualizado unos segundos.
Sesiones	Clave-valor (Redis)	Busca el token de forma directa y vence las sesiones solo con un TTL, sin proceso de limpieza.	SQL, que puede almacenarlas, pero obliga a barrer periódicamente las vencidas.	Se pierde persistencia, así que si cae el nodo se cierran sesiones, algo aceptable en este caso.

Dato	Modelo elegido	Razón y ventaja frente a SQL	Alternativa descartada	Trade-off (qué se resigna)
Pronósticos	Documental (MongoDB), con confirmación en la mayoría de las réplicas	Agrupar los pronósticos del usuario y reparte el pico de escritura particionando por id_usuario.	SQL, que aporta integridad, pero concentra cientos de millones de filas y todo el pico en un maestro.	La escritura tarda algo más por esperar la confirmación, y ese es el precio de no perder un pronóstico.
Puntaje / ranking	Sorted set (Redis) con MongoDB como fuente reconstruible	La estructura ya mantiene a los usuarios ordenados por puntos, así que sumar puntos, pedir el Top 100 o ubicar la posición propia no obliga a ordenar millones de filas en cada consulta.	SQL, que es persistente, pero cada actualización masiva reescribe el índice de puntos.	El sorted set es volátil y tiene que poder reconstruirse a partir de los pronósticos.
Comentarios	Columnas anchas (Cassandra)	Reparte la escritura entre nodos usando el partido como clave de partición y el tiempo como orden, que es justo la consulta esperada.	MongoDB o SQL, más flexibles, pero que sostienen peor un millón de escrituras por partido.	Las consultas no previstas se complican, porque buscar por otro criterio exige mantener otra tabla.
Estadísticas	Series temporales (InfluxDB)	Está pensado para consultas por intervalo, agregaciones y reducción de resolución de los datos históricos.	SQL o columnar, que almacenan las mediciones, pero dejan la consulta temporal a cargo del programador.	Solo sirve si el dato tiene componente temporal, y no modela relaciones.
Auditoría	Columnas anchas (Cassandra)	Permite escribir de forma continua y distribuir el crecimiento histórico sin degradar el resto del sistema.	SQL particionado, que mantiene las relaciones, pero con un costo de índices y backups que crece sin techo.	Las consultas exploratorias se resuelven mal, porque solo salen bien las previstas en la clave de partición.
Modelo unificado de personas	Objetos (IRIS)	Jugadores, técnicos y árbitros comparten atributos, así que una jerarquía Persona → Jugador / Técnico / Árbitro evita repetir el modelo, e IRIS permite además consultar esos mismos objetos por SQL sin duplicarlos.	MongoDB o Neo4j, que cubren documentos y grafos, pero no ofrecen herencia ni acceso dual objeto/SQL.	El diseño queda acoplado al modelo de clases, y cambiarlo obliga a migrar las instancias.

Herramientas de observabilidad:

Prometheus: Recopilar métricas de solicitudes por segundo, latencia, tasa de errores, disponibilidad y uso de recursos por motor.

Grafana: Visualiza esas métricas en paneles y permite verificar si la arquitectura cumple los objetivos de latencia y uptime.

Estas herramientas no almacenan los datos principales del fixture, sino que permiten medir y visualizar el comportamiento de la arquitectura.

4. Conclusión

Los datos del Fixture Mundial 2030 no se comportan de la misma manera. Equipos, jugadores y partidos tienen volumen fijo y relaciones claras; pronósticos, sesiones, comentarios, estadísticas y auditoría reciben millones de operaciones o crecen sin límite. El ranking agrega una tercera necesidad, que es recalcular y ordenar millones de puntajes después de cada partido respetando las posiciones compartidas.

Por eso proponemos una solución híbrida y justificada dato por dato. Usamos el modelo documental para las entidades del torneo, clave-valor para lo que exige latencia mínima, grafo para los recorridos entre entidades, columnas anchas para la escritura masiva, series temporales para las métricas en vivo y objetos para el modelo unificado de personas, mientras que Prometheus y Grafana nos permiten observar el funcionamiento del conjunto. No elegimos NoSQL porque SQL sea malo, sino porque cada patrón se resuelve de forma más natural con un motor especializado y porque en cada caso sabemos qué estamos resignando a cambio. El paso siguiente es validar esta matriz en el Hito 2 y empezar la implementación en MongoDB con equipos y jugadores.

Anexo A. Esquema relacional de referencia

Modelo que se usaría en una solución puramente relacional y sobre el que se apoya el análisis de la sección 2. En rojo, las restricciones e índices que se agregaron respecto de la primera versión.